



ADVANCED LINUX ADMINISTRATION

Sandeep Kumar Sharma

INTRODUCTION

Name : Sandeep Kr. Sharma

Total experience : 11 Years (Cloud Engineer , DevOps Engineer , Cloud Infra Engineering)

Domain – Cloud Computing (AWS, Azure, GCP) Infrastructure , DevOps

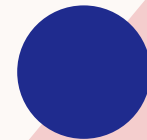
Tools Known : Git, Docker, Kubernetes , Maven, Jenkins , SonarQube, Sonar lint , Horusec , Trivy, Owasp, Anisble , Chef

**Certification : AZ104, AZ305, AWS SAA-C03, Certified Site Reliability Engineering Certification
(CSREF)**

AGENDA

- Introduction
- Advanced System Management: Package Management and System Configuration Engaging the audience
- Security and Networking
- Performance Tuning and Monitoring : Prometheus
- Ansible
- Shell Scripting

3



OPERATING SYSTEM



@ Techlander Solutions



OPERATING SYSTEM

5

- An operating system (OS) is a software that acts as an intermediary between computer hardware and the computer user.
- It provides a set of services and functionalities to manage hardware resources, facilitate communication between software applications and the hardware, and provide a user interface.



CORE FEATURES OF OPERATING SYSTEMS

6

Process Management

Memory
Management

File System
Management

Device Management

Security and
Protection

User Interface

Networking

System Calls

Error Handling

Utility Programs

Task Synchronization
and Communication



OPERATING SYSTEM PROVIDER

7



LINUX INTRODUCTION

8

- Linux is an open-source Unix-like operating system-based family on the Linux kernel, and the OS kernel was first published on 17 September 1991 by Linus Torvalds.
- Typically, Linux is packaged as the Linux distribution, which contains the supporting libraries and system software and kernel, several of which are offered by the GNU Project.
- Several Linux distributions use the term "Linux" in the title, but the Free Software Foundation uses the "GNU/Linux" title to focus on the necessity of GNU software, causing a few controversies.



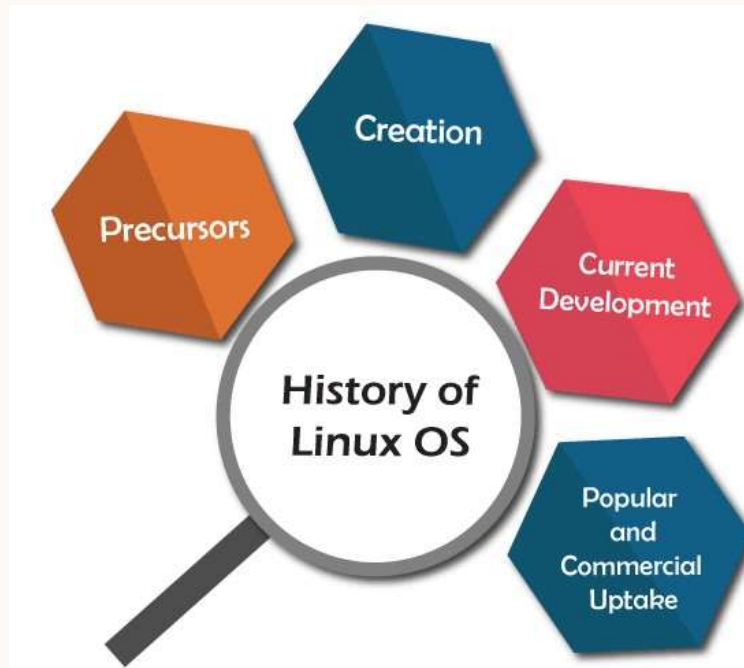
LINUX ARCHITECTURE

- **Kernel:** The heart of the UNIX Operating System. Keeps track of the disks, tapes, printers, terminals, communication lines and any other devices attached to the computer. It also interfaces between the computer's hardware and the users.
- **Shell:** A program that interfaces between the user and the UNIX Operating System. It listens to the user's terminal and translates the actions requested by the user. There are a number of different Shells that may be used.
- **Application:** Application programs, such as Word Processors, spreadsheets and Database Management Systems, may be installed alongside the UNIX Commands. A user may run an utility or application through the shell.



HISTORY OF LINUX OS

10



HISTORY OF LINUX OS

11

- **Precursors (1969-1973)**
 - Unix-based operating system conceived at AT&T's Bell Labs.
 - Initially written in assembly language, later updated to C language by Dennis Ritchie in 1973.
- **Creation (1991)**
 - Linus Torvalds registered in a Unix course at the University of Helsinki in the 1990s.
 - Frustrated by MINIX licensing, Torvalds started developing his own kernel, which became the Linux Kernel.



HISTORY OF LINUX OS

12

- Linux Development
 - Linux kernel development initiated on MINIX.
 - GNU applications replaced MINIX components, fostering the development of a free and fully functional operating system.
- Current Development
 - Greg Kroah-Hartman and William John Sullivan leads Linux Kernel development.



HISTORY OF LINUX OS

13

Popular and Commercial Uptake

- Linux adoption in the mid-1990s in supercomputing (NASA).
- IBM, Dell, and HP provided Linux support, challenging Microsoft's desktop OS monopoly.
- Used from embedded systems to supercomputers; widespread use in server installations (LAMP stack).
- Growth in enterprise and home desktops.
- Linux distributions popular in the netbook market, including Google's ChromeOS.
- Executive director of the Free Software Foundation, supports GNU components.



LINUX ADOPTION TIMELINE

14

- **1998:** Many major companies such as IBM, Compaq and Oracle announce their support for Linux. The Cathedral and the Bazaar is first published as an essay (later as a book), resulting in Netscape publicly releasing the source code to its Netscape Communicator web browser suite. Netscape's actions and crediting of the essay[55] brings Linux's open source development model to the attention of the popular technical press. In addition a group of programmers begins developing the graphical user interface KDE.
- **1999:** A group of developers begin work on the graphical environment GNOME, destined to become a free replacement for KDE, which at the time, depends on the, then proprietary, Qt toolkit. During the year IBM announces an extensive project for the support of Linux.
- **2000:** Dell announces that it is now the No. 2 provider of Linux-based systems worldwide and the first major manufacturer to offer Linux across its full product line.
- **2002:** The media reports that "Microsoft killed Dell Linux"



LINUX ADOPTION TIMELINE

- **2004:** The XFree86 team splits up and joins with the existing X standards body to form the X.Org Foundation, which results in a substantially faster development of the X server for Linux.
- **2005:** The project openSUSE begins a free distribution from Novell's community. Also the project OpenOffice.org introduces version 2.0 that then started supporting OASIS OpenDocument standards.
- **2006:** Oracle releases its own distribution of Red Hat Enterprise Linux. Novell and Microsoft announce cooperation for a better interoperability and mutual patent protection.
- **2007:** Dell starts distributing laptops with Ubuntu pre-installed on them.
- **2009:** Red Hat's market capitalization equals Sun's, interpreted as a symbolic moment for the "Linux-based economy."



LINUX ADOPTION TIMELINE

16

- **2011:** Version 3.0 of the Linux kernel is released.
- **2012:** The aggregate Linux server market revenue exceeds that of the rest of the Unix market.
- **2013:** Google's Linux-based Android claims 75% of the smartphone market share, in terms of the number of phones shipped.
- **2014:** Ubuntu claims 22,000,000 users.
- **2015:** Version 4.0 of the Linux kernel is released.
- **2019:** Version 5.0 of the Linux kernel is released. New features such as AMD FreeSync display support, Raspberry Pi Touchscreen support, Btrfs swap file support and Adiantum data encryption make into the [kernel](#).



LINUX DISTRIBUTIONS

- A Linux distribution is a collection of (usually open source) software on top of a Linux kernel.
- A distribution can bundle server software, system management tools, documentation and many desktop applications in a central secure software repository. A distribution aims to provide a common look and feel, secure and easy software management and often a specific operational purpose.

Ubuntu	Debian	Fedora
CentOS	openSUSE	Mint
Kali Linux	Red Hat Enterprise Linux (RHEL)	Alpine Linu



LINUX DISTRIBUTIONS

18

- **Red Hat**

Red Hat is a billion dollar commercial Linux company that puts a lot of effort in developing Linux. They have hundreds of Linux specialists and are known for their excellent support. They give their products (Red Hat Enterprise Linux and Fedora) away for free. While Red Hat Enterprise Linux (RHEL) is well tested before release and supported for up to seven years after release, Fedora is a distro with faster updates but without support.

- **Ubuntu**

Canonical started sending out free compact discs with Ubuntu Linux in 2004 and quickly became popular for home users (many switching from Microsoft Windows). Canonical wants Ubuntu to be an easy to use graphical Linux desktop without need to ever see a command line. Of course they also want to make a profit by selling support for Ubuntu.



LINUX DISTRIBUTIONS

19

- **Debian**

There is no company behind Debian. Instead there are thousands of well organised developers that elect a Debian Project Leader every two years. Debian is seen as one of the most stable Linux distributions. It is also the basis of every release of Ubuntu. Debian comes in three versions: stable, testing and unstable. Every Debian release is named after a character in the movie Toy Story.

- **CentOS**

CentOS, short for Community ENTERprise Operating System, is a Linux distribution that aims to provide a free, open-source, and community-supported alternative to the commercially available Red Hat Enterprise Linux (RHEL). CentOS is built from the same upstream source code as RHEL, and it strives to maintain binary compatibility with RHEL.



CHOOSING A DISTRIBUTION

Choosing a Linux distribution will always be based on a company or individual choice but here are some name reason(s) which can help decide

Distribution Name	Reason
Red Hat Enterprise (RHEL)	Good Support Contract
Centos	You want Red Hat without the support contract from Red Hat.
Fedora	Need it on Laptop/Desktop
Linux Mint	Graphical desktop to play music, watch movies
Debian/Ubuntu	Servers, laptops
Kali	For Ethical Hacking

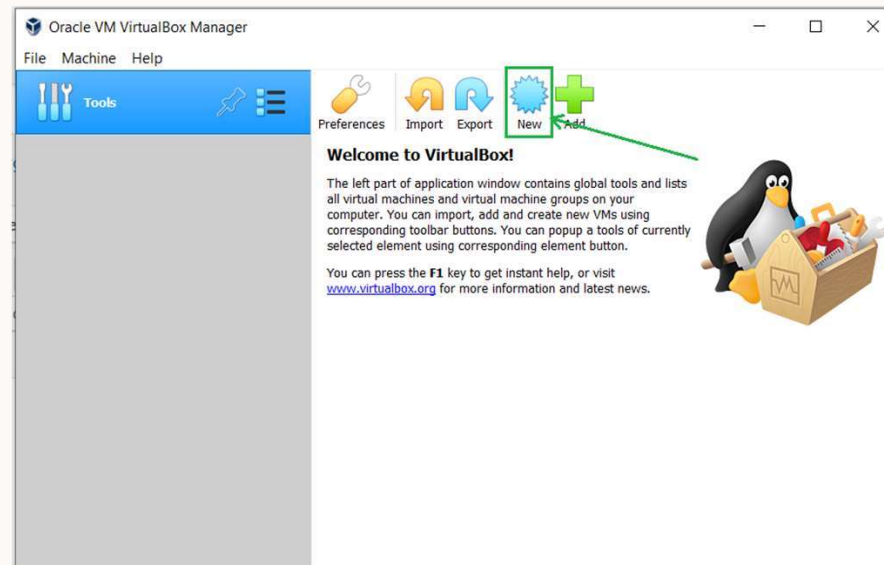


HOW TO INSTALL UBUNTU

Steps to Install Ubuntu on VirtualBox

Step 1: Before we begin with the installation process, we need to Download ISO for Ubuntu. For that, all the versions of Ubuntu are available on the Official Site.

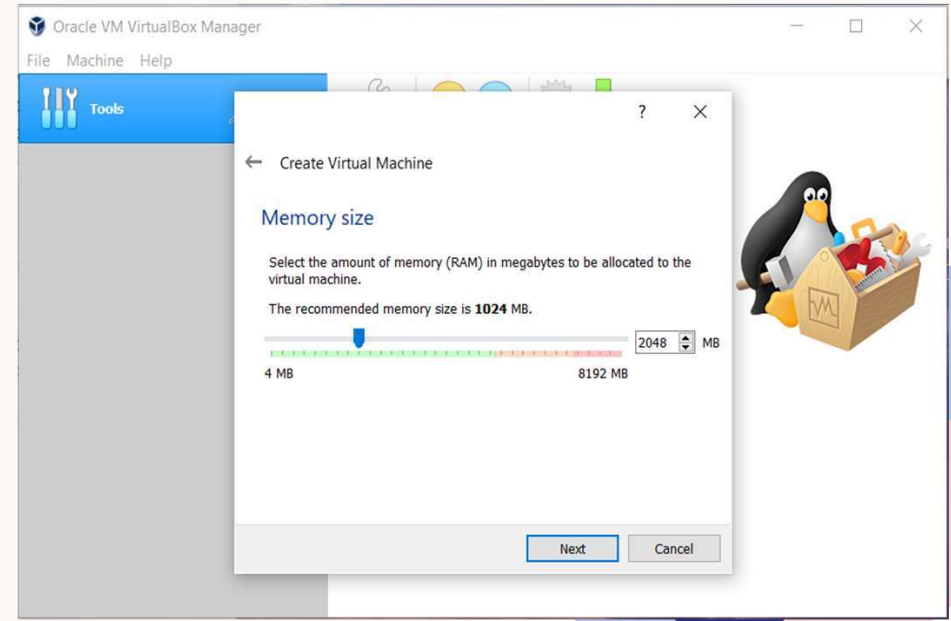
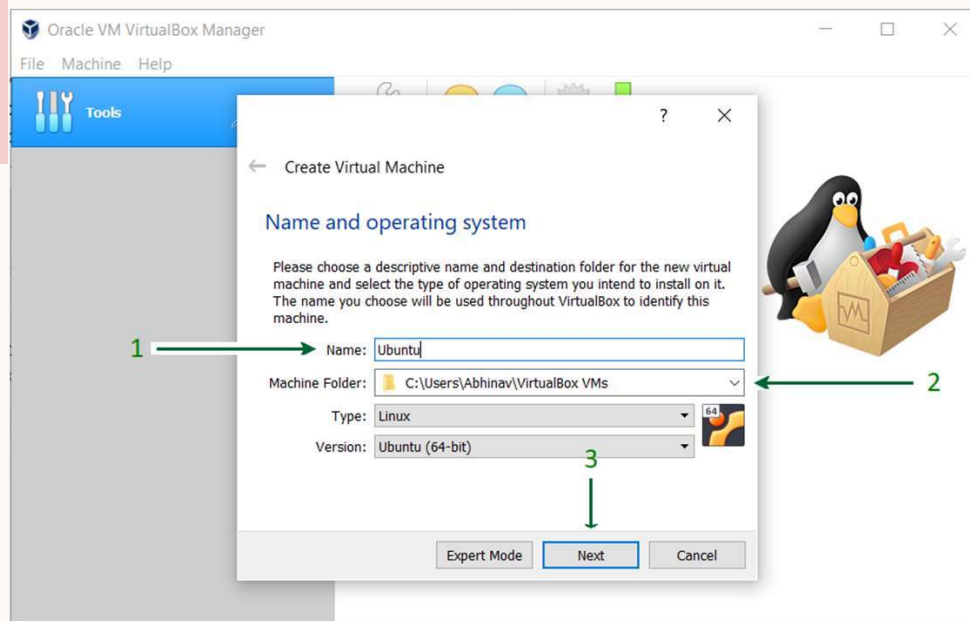
Step 2: Open VirtualBox and click on the New button.



HOW TO INSTALL UBUNTU

22

Step 3 & 4 : Give a Name to your Virtual Machine and select the Location for it to install and Assign RAM Size to your Virtual Machine.



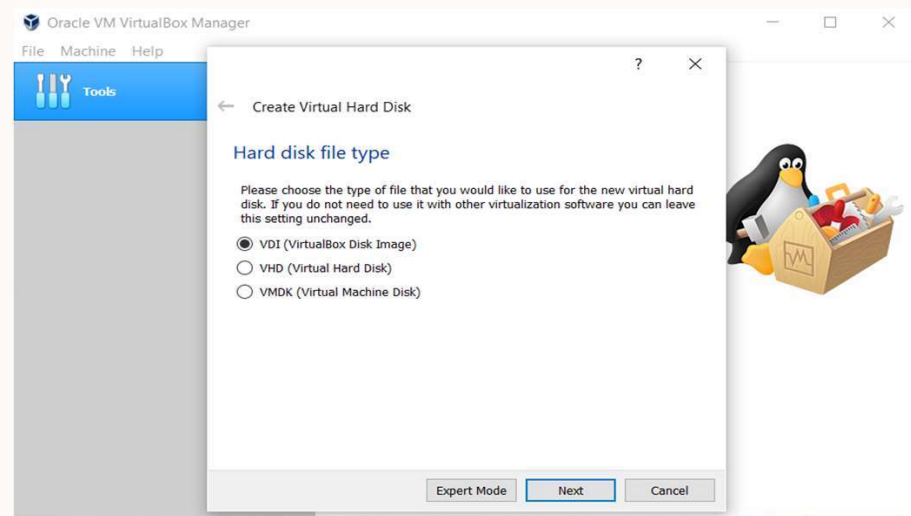
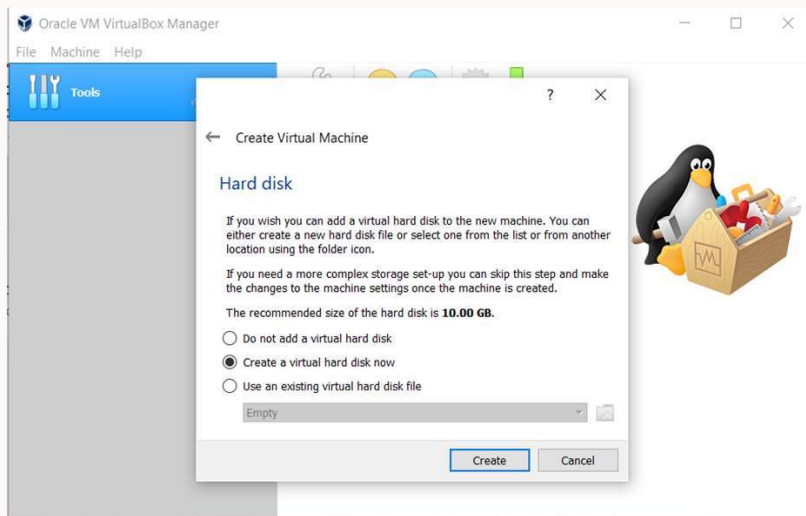
@ Techlander Solutions



HOW TO INSTALL UBUNTU

23

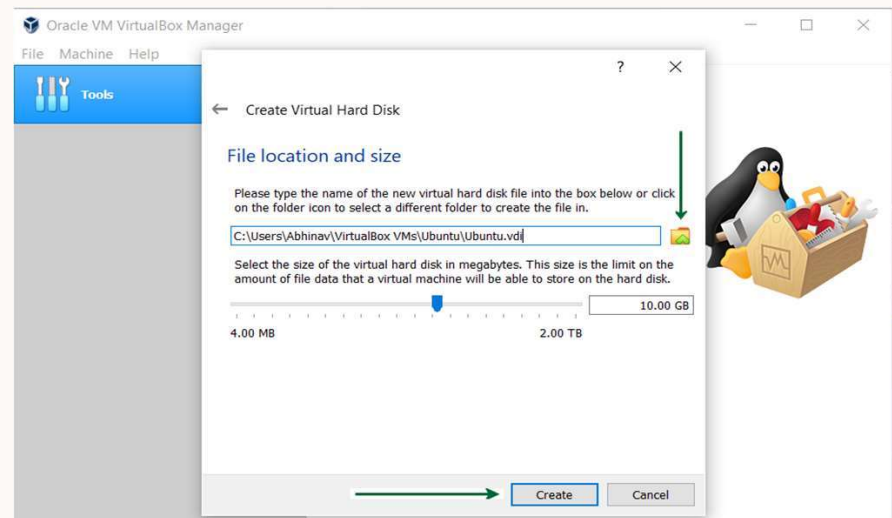
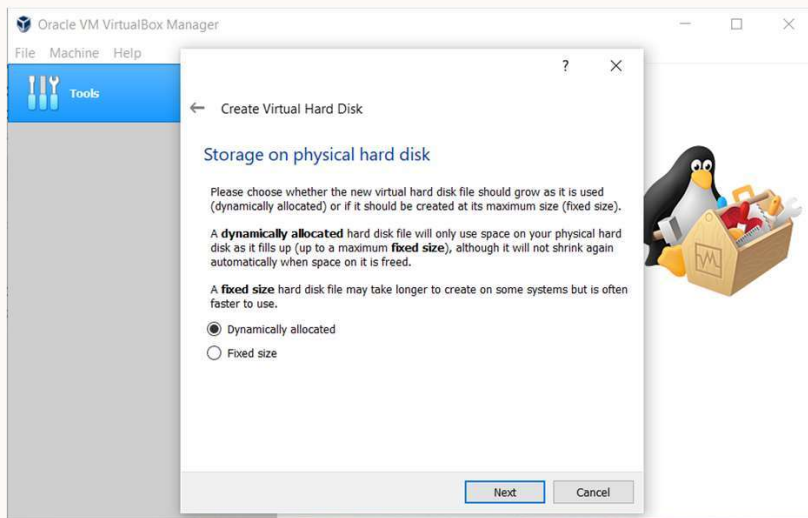
Step 5: Give a Name to your Virtual Machine and select the Location for it to install and Assign RAM Size to your Virtual Machine.



HOW TO INSTALL UBUNTU

24

Step 6: set the physical storage and RAM



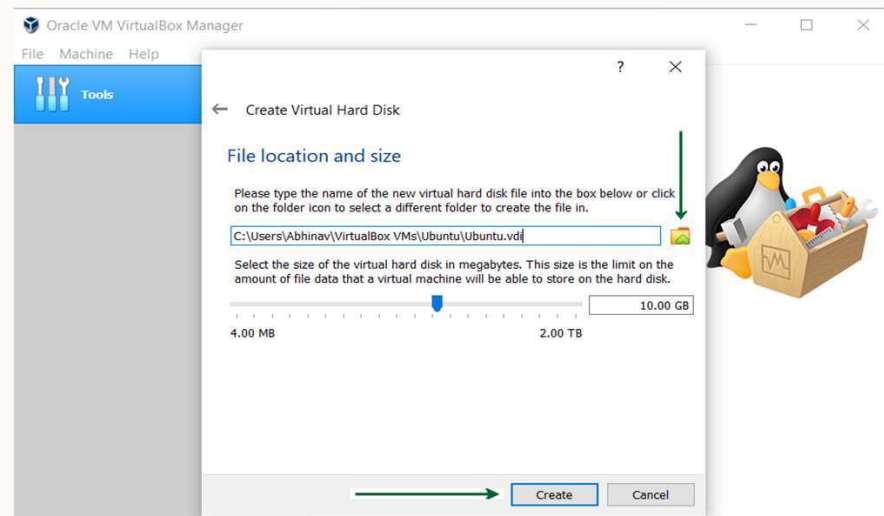
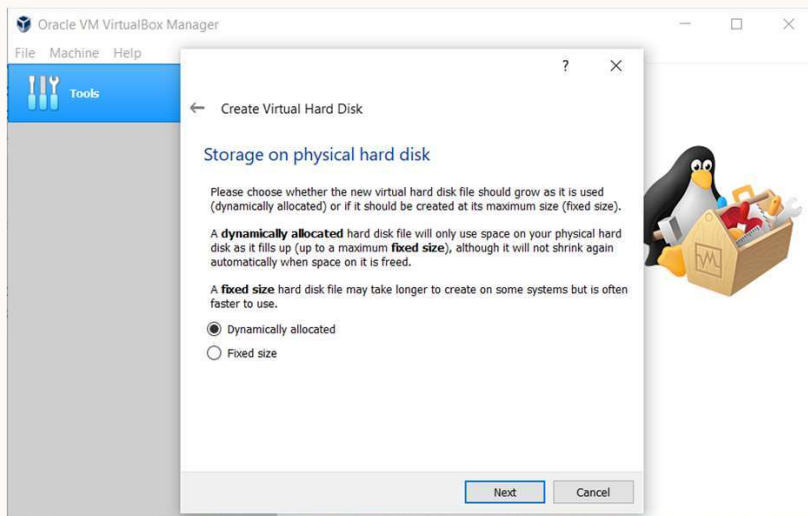
@ Techlander Solutions



HOW TO INSTALL UBUNTU

25

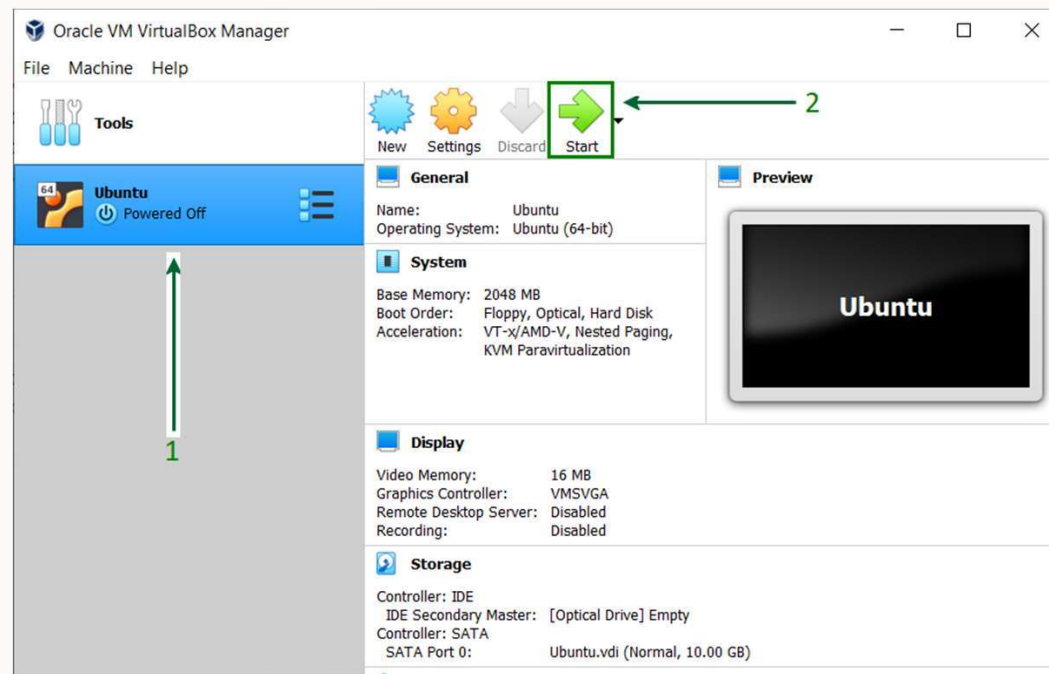
Step 6: set the physical storage and RAM



HOW TO INSTALL UBUNTU

26

Step 7 : start VM and configure it



@ Techlander Solutions



INTRODUCTION TO LINUX TERMINAL

1. The Linux terminal is also known as **command-line**, **console**, or **shell**. It is a text interface for our Computer. We can interpret the commands and also write our scripts with this system program.
2. It might look difficult at first but once we get familiar to do it, it will be easy to use.
3. We can manually execute commands by typing in the terminal with the Linux command line. We can also automate commands with the use of **Shell Scripts**.

Note: All the distributions of Linux come with the command-line. The Linux terminal provides us an interface in which we could input commands and check the result as text. We could use the terminal for accomplishing the task such as navigating a territory or moving files without using a GUI. These terminals only transfer information. We require a shell for the OS to know this information.



THE SHELL

- The shell can be defined as an interface of a command-line that interprets the script files and commands of a user in the Linux system and tells the operating system of the server how to act with them. Several shells are available that are extensively used like C shell (csh) and Bourne shell (sh).
- All the shells have their set of features and intricacies related to how commands can be interpreted, but these shells feature output and input redirection, condition-testing, variables, among other things.
- Usually, the Bourne-Again shell is known as bash. It is a default shell for almost all the distributions of Linux including RedHat, CentOS, and Ubuntu.





BASIC LINUX NAVIGATION COMMANDS

WHAT ARE COMMANDS

- A command is an instruction given to our computer by us to do whatever we want. In Mac OS, and Linux it is called terminal, whereas, in windows it is called command prompt. Commands are always case sensitive.
- Commands are executed by typing in at the command line followed by pressing enter key.
- This command further passes to the shell which reads the command and execute it. Shell is a method for the user to interact with the system. Default shell in Linux is called bash (Bourne-Again Shell).
- There are two types of shell commands:
- Built-in shell commands: They are part of a shell. Each shell has some built in commands.
- External/Linux commands: Each external command is a separate executable program written in C or other programming languages.



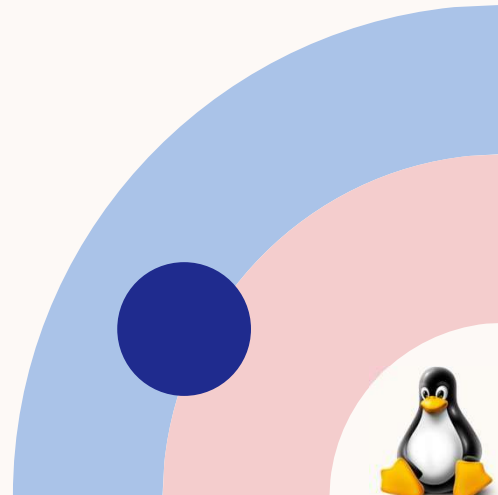
ls COMMAND IN LINUX

- The ls command is commonly used to identify the files and directories in the working directory. This command is one of the many often-used Linux commands that you should know.
- This command can be used by itself without any arguments and it will provide us the output with all the details about the files and the directories in the current working directory. There is a lot of flexibility offered by this command in terms of displaying data in the output. Check the below image for the output.



PWD COMMAND IN LINUX

- The pwd command is mostly used to print the current working directory on your terminal. It is also one of the most commonly used commands.
- Now, your terminal prompt should usually include the entire directory. If it doesn't, this is a quick command to see which directory you're in. Another purpose for this command is when creating scripts because it can help us find the directory in which the script was saved. The below pictures are the output with the command.



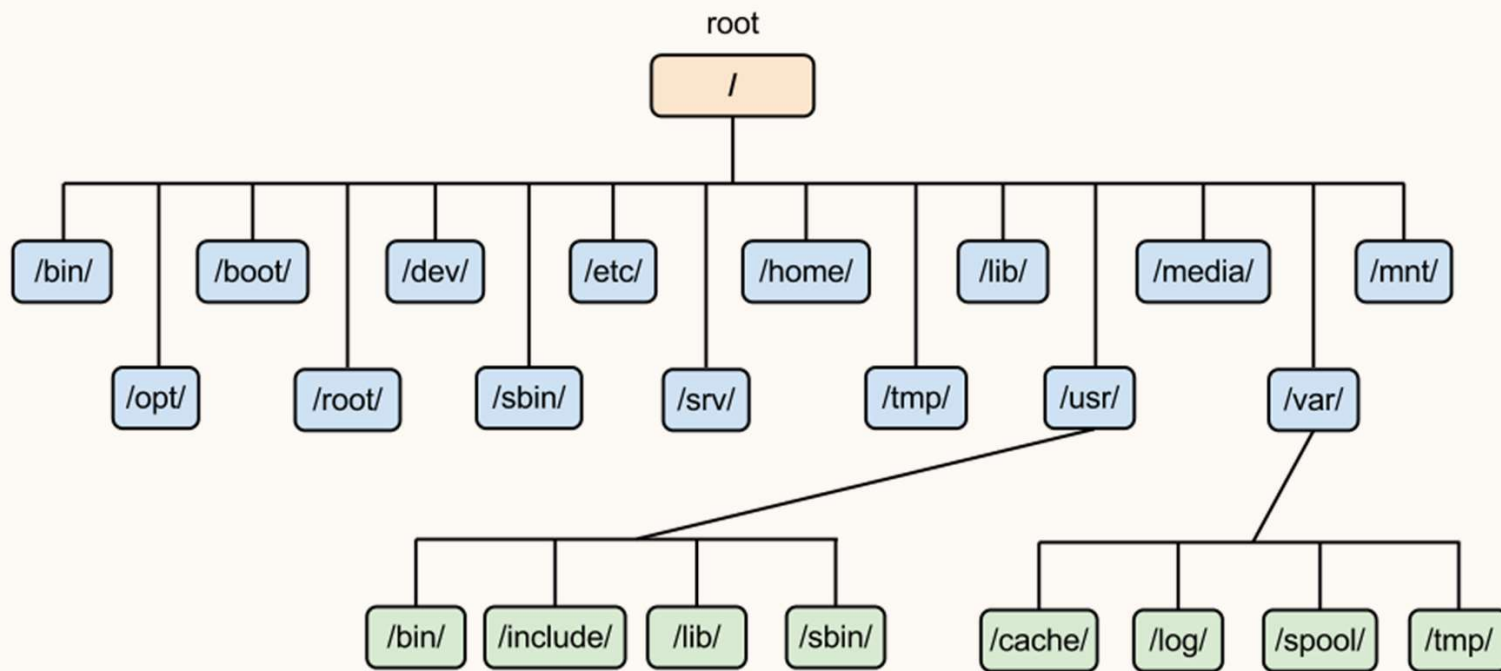
MKDIR COMMAND

- This allows you to create fresh directories in the terminal itself.
- The default syntax is `mkdir <directory name>` and the new directory will be created.
- For example, if you want to create a directory as “sandeep-demo” then the basic syntax would be:
- `mkdir sandeep-demo`
- In case you want to create another directory inside the main directory sandeep-demo to store projects, you can use the following command to do so. `mkdir sandeep-demo /project1`



LINUX FILE SYSTEM

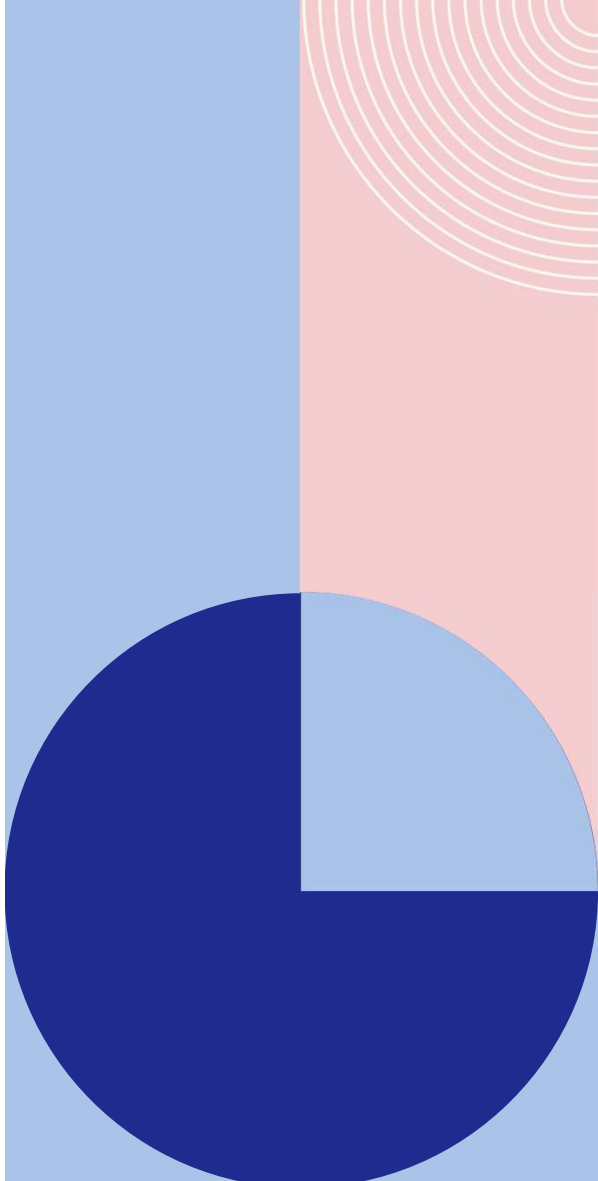
34



LAB EXERCISE

1. Use the "ls" command to list directory contents.
2. Experiment with different options such as -l and -a.
3. Execute "pwd" to display the present working directory.
4. Change directories using "cd" and verify changes with "pwd."
5. Create a new directory and confirm its location.
6. Create a new directory using "mkdir."
7. Confirm the successful creation with "ls."
8. Confirm your location with "pwd" after each navigation.
9. Create empty directories.
10. Use "rmdir" to remove one directory.
11. Confirm deletion with "ls" and "rmdir" again.





ADVANCED PACKAGE MANAGEMENT

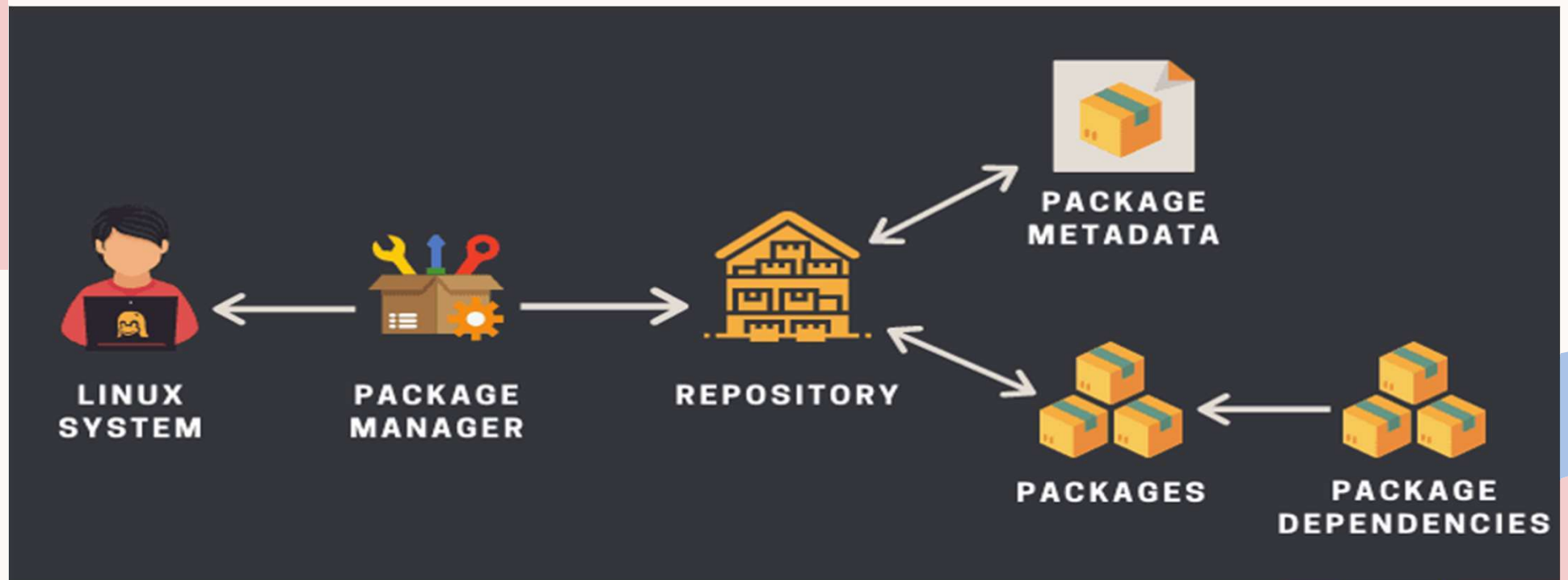
PACKAGE MANAGEMENT SYSTEM

- Package management in Linux refers to the process of handling software packages—collections of files and metadata—that can be installed, updated, configured, and removed on a Linux system.
- A package manager is a set of tools and utilities designed to automate these tasks, making it easier for users to manage software on their systems.
- The package management system is an integral part of maintaining a Linux distribution and ensures the proper installation, updating, and removal of software components.



ADVANCED PACKAGE MANAGEMENT

38



CORE CONCEPTS FOR PM

1. Packages

- A package is a compressed archive that contains files, metadata, and instructions for installing or removing software on a system.

2. Repositories

- A repository is a collection of packages and metadata, often hosted online. Package managers use repositories to download and install software

3. Dependencies

- A dependency is a software component required by another software to function properly. Package managers handle dependencies to ensure that all required components are installed.

4. Package Managers

- A package manager is a tool or set of tools that automates the process of installing, updating, configuring, and removing software packages on a system.



FUNCTIONS OF PACKAGE MANAGER

The *package management system* contains some typical *functions* which are mentioned below:

- Dealing with the file archivers for extracting package archives.
- Assuring the authenticity and integrity of the package by authenticating their digital certificates and checksums respectively.
- Updating, installing, downloading, or looking up existing software through an app store or software repository.
- Combining packages via function for reducing the confusion of the user.
- Maintaining dependencies for ensuring a package has been installed along with every package it needs. So, ignoring "**dependency hell**".



PACKAGE MANAGERS

There are lots of package managers in Linux, each working a bit differently. Some of the package managers are

1. APT (Advanced Package Tool)
2. YUM (Yellowdog Updater, Modified)
3. DNF (Dandified YUM)
4. ZYPPER
5. PACMAN
6. PORTAGE
7. SLACKWARE PACKAGE MANAGEMENT



APT (ADVANCED PACKAGE TOOL)

APT, or Advanced Package Tool, is a package management system used in Debian-based Linux distributions, such as Debian itself, Ubuntu, and others. It is a set of tools that work together to simplify the process of managing software packages on a Linux system. APT is commonly used to install, upgrade, and remove software packages, handling dependencies automatically.

- **Distributions:** Ubuntu, Debian, and Kali Linux
- **Commands:** apt, apt-get, apt-cache
- **Underlying package management tool:** dpkg
- **Package file format:** .deb



KEY APT COMMANDS

Update Package Lists: This command updates the local package index, which is a database of available packages.

```
sudo apt-get update
```

Upgrade Installed Packages: This command upgrades the installed packages to their latest versions.

```
sudo apt-get upgrade
```

Install a Package: Use this command to install a new package.

```
sudo apt-get install package_name
```

Remove a Package: This command removes a package without removing its configuration files.

```
sudo apt-get remove package_name
```

Remove a Package with Configuration Files: Use this command to completely remove a package, including its configuration files.

```
sudo apt-get purge package_name
```



KEY APT COMMANDS

Search for Packages: Search for packages by name or description.

```
apt-cache search search_term
```

Show Package Information: Display detailed information about a package.

```
apt-cache show package_name
```

Autoremove Unused Packages: Remove packages that were automatically installed but are no longer required.

```
sudo apt-get autoremove
```

List Installed Packages: Display a list of installed packages.

```
dpkg -l
```



YUM (YELLOWDOG UPDATER, MODIFIED)

YUM, or Yellowdog Updater, Modified, is a package management utility for RPM-based Linux distributions, such as Red Hat Enterprise Linux (RHEL), CentOS, and Fedora. It simplifies the process of installing, updating, and removing software packages on a system. YUM is known for its dependency resolution capabilities, making it easy to manage software and ensure that all required components are installed.

- **Distributions:** RHEL/CentOS 7, Fedora 21, and earlier versions of both distributions
- **Command:** yum
- **Underlying package management tool:** RPM (*RPM Package Manager*)
- **Package file format:** .rpm



KEY YUM COMMANDS

Update Package Lists: This command updates the local package index, which is a database of available packages.

```
sudo yum update
```

Upgrade Installed Packages: This command upgrades the installed packages to their latest versions.

```
sudo yum upgrade
```

Install a Package: Use this command to install a new package.

```
sudo yum package_name
```

Remove a Package: This command removes a package without removing its configuration files.

```
Sudo yum remove package_name
```



KEY YUM COMMANDS

Search for Packages: Search for packages by name or description.

```
yum search search_term
```

Clean Package Cache: Remove cached package files to free up disk space.

```
Sudo yum clean package
```

Autoremove Unused Packages: Remove packages that were automatically installed but are no longer required.

```
sudo yum autoremove
```



DNF - DANDIFIED YUM

DNF, or Dandified YUM, is a package management utility and the default package manager for modern Red Hat-based Linux distributions, including Fedora, RHEL (Red Hat Enterprise Linux) 8 and later, and CentOS 8 and later. DNF is the next-generation replacement for YUM (Yellowdog Updater, Modified) and provides improved performance, better dependency resolution, and a more user-friendly interface.

- Distributions:** RHEL/CentOS 8, Fedora 22, and later versions of both distributions
- Commands:** dnf, yum
- Underlying package management tool:** RPM (*RPM Package Manager*)
- Package file format:** .rpm



DNF - DANDIFIED YUM

- DNF provides improved performance and dependency resolution compared to its predecessor YUM.
- The syntax and options for DNF are very similar to YUM, making it easier for users familiar with YUM to transition.
- DNF also has a modular architecture that allows for more flexibility and extensibility.
- Always refer to the man pages (`man dnf`, `man dnf.conf`) for more detailed information and options.
- DNF is a modern and efficient package management tool for RPM-based distributions. It simplifies the software management process by automating common tasks and handling dependencies effectively.



ZYPPER

Zypper is the package management tool used in openSUSE and SUSE Linux Enterprise distributions.

It is designed to simplify the process of installing, updating, and managing software packages on the system. zypper handles dependencies, resolves conflicts, and provides a command-line interface for package management tasks.

- **Distributions:** openSUSE
- **Command:** zypper
- **Underlying package management tool:** ZYpp (also called libzypp)
- **Package file format:** .rpm



ZYPPER

- ✓ **zypper** uses the concept of repositories to manage software packages. Repository configuration files are typically located in `/etc/zypp/repos.d/`.
- ✓ The syntax and options for **zypper** are designed to be user-friendly and similar to other package management tools.
- ✓ **zypper** supports both traditional package management and the concept of "patterns" that represent groups of packages related to specific tasks or roles.
- ✓ Always refer to the man pages (**man zypper**) for more detailed information and options.
- ✓ **zypper** is a powerful and versatile package management tool that simplifies the process of managing software on openSUSE and SUSE Linux Enterprise systems. It provides an easy-to-use command-line interface for performing various package management tasks.



PACMAN

52

Pacman is the package management utility used in Arch Linux and its derivatives. It is designed to simplify the process of installing, updating, and managing software packages on Arch-based systems. Pacman handles dependencies, resolves conflicts, and provides a command-line interface for package management tasks.

- **Distributions:** Arch-based, including Arch and Manjaro
- **Command:** pacman
- **Package file format:** .tar.xz (and other compressed tar formats)



PORTAGE

Portage, the package manager for Gentoo, is quite a bit different than other solutions. Instead of using pre-compiled binary packages, Portage packages are typically shell scripts called ebuilds. The emerge CLI tool can run these shell scripts to install packages, and is also responsible for managing dependencies and a database of installed packages.

- **Distributions:** Gentoo
- **Command:** emerge
- **Package file format:** ebuild shell script or .tbz2 (compressed tar archive)



SLACKWARE PACKAGE MANAGEMENT

Slackware comes bundled with a few package management tools. The `pkgtool` is a TUI (menu-driven text interface) for managing packages and installing local packages. To install packages located on the internet, the `slackpkg` tool can be used. For more advanced tasks, use specialized tools like `installpkg`, `upgradepkg`, and `removepkg`.

Distributions: Slackware

Commands: `slackpkg`, `pkgtool`, `installpkg`, `upgradepkg`, `removepkg`

Package file format: `.tgz` and `.txz` (compressed tar archive)



COMPILING SOFTWARE PACKAGES

55

We have many open source and commercial software packages installed on our systems, which are available to all users. However, users are allowed to download and install small software packages in their own home directory, software usually only useful to them personally.

There are a number of steps needed to install the software.

- Locate and download the source code (which is usually compressed)
- Unpack the source code
- Compile the code
- Install the resulting executable
- Set paths to the installation directory



COMPILING SOURCE CODE

All high-level language code must be converted into a form the computer understands. For example, C language source code is converted into a lower-level language called assembly language. The assembly language code made by the previous stage is then converted into object code which consists of fragments of code which the computer understands directly. The final stage in compiling a program involves linking all of the fragments of object code together into a single executable program.

Performing all of these steps by hand is complicated and beyond the capability of the ordinary user. A number of utilities and tools have been developed for programmers and end-users to simplify these steps.



MAKE AND THE MAKEFILE

- The **make** command allows programmers to manage the compilation of large programs or groups of programs. It aids in developing large programs by keeping track of which portions of the entire program have been changed, compiling only those parts of the program which have changed since the last compile.
- The **make** program gets its set of compile rules from a text file named Makefile which resides in the same directory as the source files. It contains information on how to compile the software, e.g. the optimization level, whether to include debugging information in the executable. It also contains information on where to install the compiled binaries (executables), manual pages, data files, dependent library files, configuration files, etc.

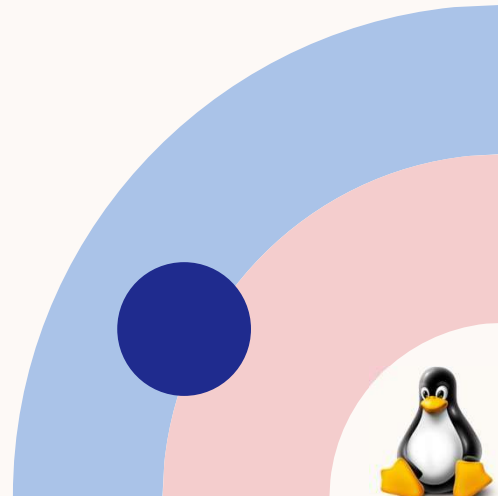


KERNEL

58

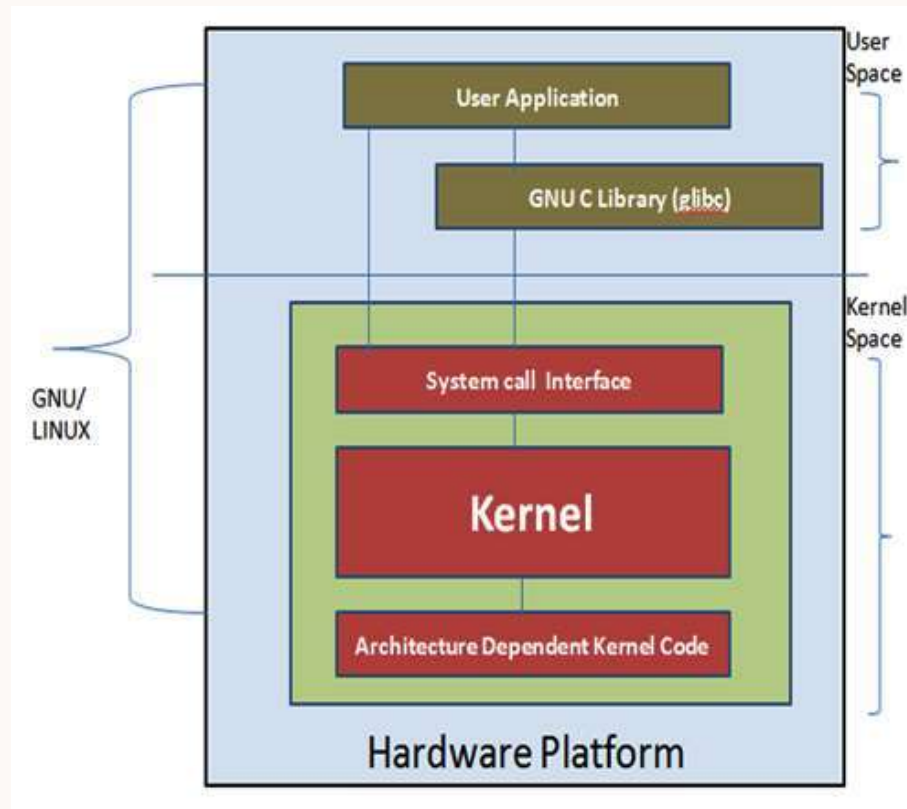
A kernel is the critical component of an operating system. It works as a bridge between the applications and data processing at the hardware level with the help of its interprocess communication and system calls.

Whenever an operating system is loaded into memory, firstly, the kernel is loaded and stays there till the operating system gets shut down. The kernel is responsible for taking care of low-level tasks such as task management, memory management, risk management, etc.



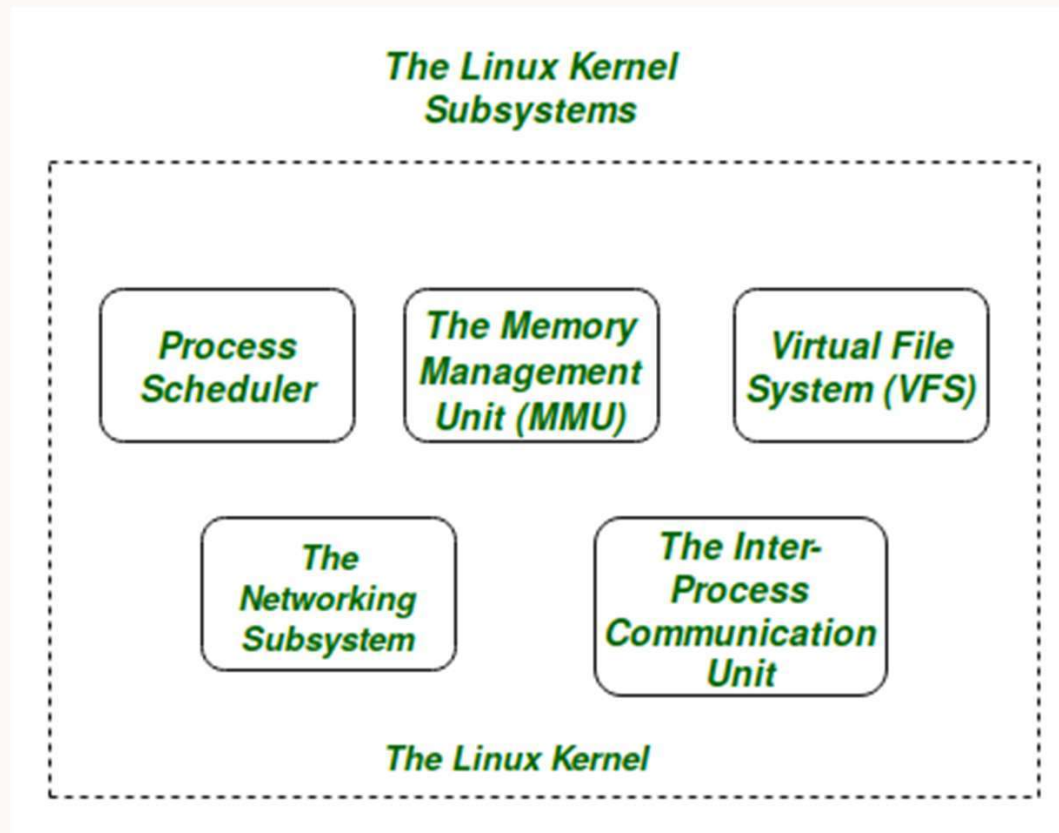
LINUX OS ARCHITECTURE

59



MAIN SUBSYSTEMS OF KERNEL

60



TYPES OF KERNEL

There are generally three types of kernels: monolithic kernels, microkernels, and hybrid kernels. Each type has its own design philosophy and characteristics. Here's an overview of each:

1. **Monolithic Kernels**
2. **Microkernels**
3. **Hybrid Kernels**



MONOLITHIC KERNELS

Definition: In a monolithic kernel, the entire operating system runs as a single program in kernel mode.

Characteristics:

- All kernel services and device drivers operate in the same address space.
- It provides fast communication between components since they are all part of the same program.
- Examples of monolithic kernels include Linux and older versions of Unix.



MICROKERNELS

Definition: In a microkernel architecture, the kernel is minimal, and most services traditionally provided by the kernel are implemented as separate processes or user-level programs.

Characteristics:

- Only essential functionality is included in the kernel.
- Additional services, like device drivers and file systems, run in user space.
- Communication between components may be slower due to inter-process communication.
- Examples include MINIX and early versions of the Mach kernel.



HYBRID KERNELS:

Definition: Hybrid kernels aim to combine the best features of monolithic and microkernels.

Characteristics:

- Some services run in the kernel space for efficiency, while others run in user space for modularity.
- Offers a compromise between the simplicity of microkernels and the performance of monolithic kernels.
- Examples include the Windows NT kernel and some versions of the XNU kernel used in macOS.



KERNEL RELATED COMMANDS

65

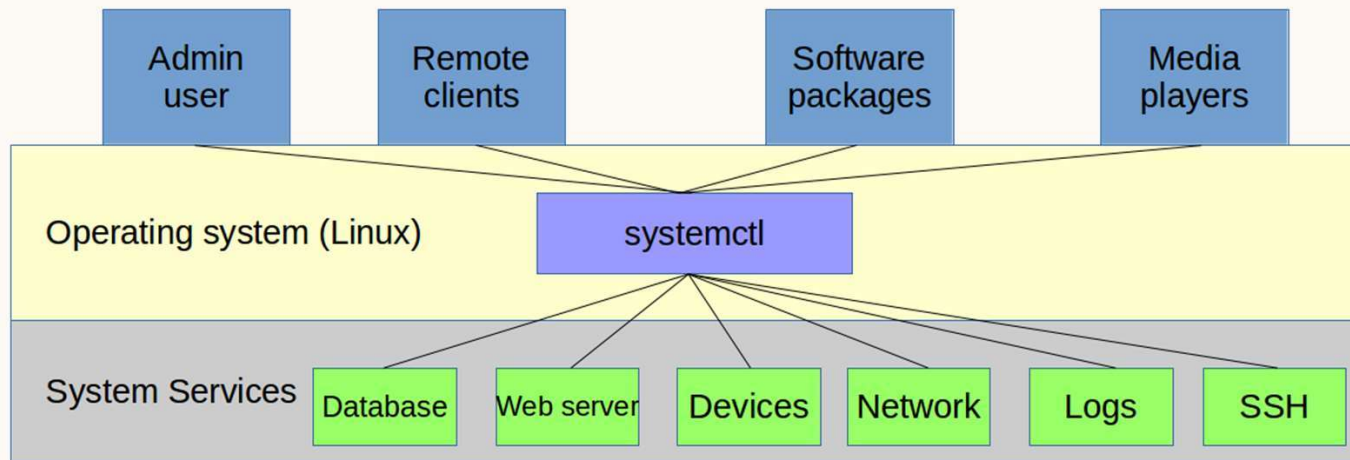
Command	Description	Example
uname	Displays basic system information, including the kernel version.	<code>uname -a</code>
dmesg	Displays kernel ring buffer messages, including boot messages.	<code>dmesg</code>
lsmod	Lists currently loaded kernel modules.	<code>lsmod</code>
modinfo	Displays information about a kernel module.	<code>modinfo usbhid</code>
<code>ls /lib/modules</code>	Lists installed kernel versions.	<code>ls /lib/modules</code>
lsblk	Lists block devices and their associated information.	<code>lsblk</code>



SYSTEMD

66

Systemd is a system and service manager for Linux operating systems. It plays a central role in initializing and managing system processes, including services. Service configuration in systemd involves defining how services should be started, stopped, and managed.



KEY ROLES OF SYSTEMD IN LINUX

1. Init System:

Role: Manages the initialization process of the system during boot.

Responsibility: Coordinates the execution of user and system-defined tasks, ensuring a smooth and efficient startup.

2. Service Manager:

Role: Controls the execution and management of services and daemons on the system.

Responsibility: Handles service activation, deactivation, status monitoring, and process supervision.

3. Dependency Management:

Role: Resolves and manages inter-service dependencies during startup and runtime.

Responsibility: Ensures that services are started in the correct order based on their dependencies, improving system reliability.

4. Parallelization:

Role: Allows for parallel execution of startup processes, leading to faster boot times.

Responsibility: Optimizes system initialization by running independent tasks concurrently, taking advantage of modern hardware capabilities.



KEY ROLES OF SYSTEMD IN LINUX

5. Logging and Journaling:

Role: Collects and manages system logs.

Responsibility: Implements the Journal, a centralized and structured logging system, providing efficient log storage, retrieval, and analysis.

6. Resource Management:

Role: Controls resource limits, permissions, and capabilities of services.

Responsibility: Allows administrators to set resource constraints (e.g., CPU, memory) for services and manage service execution permissions.

7. System and User Sessions:

Role: Manages system-level and user-level sessions.

Responsibility: Coordinates the setup and teardown of user sessions, handling user-specific services and processes.

8. Security Features:

Role: Implements various security measures.

Responsibility: Enforces security policies, provides sandboxing capabilities, and supports technologies like cgroups and namespaces for process isolation.



KEY ROLES OF SYSTEMD IN LINUX

9. Networking:

Role: Manages network-related tasks and configurations.

Responsibility: Handles network device setup, hostname management, and network time synchronization.

10. Target Units:

Role: Defines sets of services and other units that are started together.

Responsibility: Allows users to define and manage system states (targets) to specify which services should be active.

11. Dynamic Device Management:

Role: Manages devices dynamically during runtime.

Responsibility: Provides udev integration for automatic detection and management of hardware devices.

12. Container Support:

Role: Supports containerization technologies.

Responsibility: Enables the integration of containers through features like systemd-nspawn, facilitating system and service isolation.

13. User Session Tracking:

Role: Monitors user sessions and login/logout events.

Responsibility: Helps track user activities, facilitating resource cleanup upon user logout.



SYSTEMCTL COMMAND

Command	Description	Example
<code>systemctl start <service></code>	Start a service	<code>sudo systemctl start apache2</code>
<code>systemctl stop <service></code>	Stop a service	<code>sudo systemctl stop apache2</code>
<code>systemctl restart <service></code>	Restart a service	<code>sudo systemctl restart apache2</code>
<code>systemctl reload <service></code>	Reload configuration of a service	<code>sudo systemctl reload apache2</code>
<code>systemctl enable <service></code>	Enable a service to start on boot	<code>sudo systemctl enable apache2</code>
<code>systemctl disable <service></code>	Disable a service from starting on boot	<code>sudo systemctl disable apache2</code>
<code>systemctl status <service></code>	Show the status of a service	<code>sudo systemctl status apache2</code>
<code>systemctl is-active <service></code>	Check if a service is active	<code>sudo systemctl is-active apache2</code>
<code>systemctl is-enabled <service></code>	Check if a service is enabled	<code>sudo systemctl is-enabled apache2</code>
<code>systemctl is-failed <service></code>	Check if a service has failed	<code>sudo systemctl is-failed apache2</code>
<code>systemctl list-units</code>	List all active units (services, targets)	<code>sudo systemctl list-units --type=service</code>
<code>systemctl list-unit-files</code>	List all unit files and their states	<code>sudo systemctl list-unit-files --type=service</code>
<code>systemctl daemon-reload</code>	Reload systemd manager configuration	<code>sudo systemctl daemon-reload</code>
<code>systemctl show <service></code>	Show detailed information about a service	<code>sudo systemctl show apache2</code>
<code>systemctl cat <service></code>	Display contents of a service unit file	<code>sudo systemctl cat apache2</code>
<code>systemctl edit <service></code>	Edit the service unit file in a text editor	<code>sudo systemctl edit apache2</code>
<code>systemctl reset-failed</code>	Reset the failed state of all units	<code>sudo systemctl reset-failed</code>
<code>systemctl list-dependencies <service></code>	List units required or wanted by a service	<code>sudo systemctl list-dependencies apache2</code>
<code>systemctl set-property <service></code>	Set runtime properties for a service	<code>sudo systemctl set-property apache2 CPUShares=500</code>



SECURITY IN LINUX

Security in Linux is a multi-faceted concept involving various layers of protection to safeguard the operating system, applications, and data from unauthorized access, attacks, and potential vulnerabilities.

Here are key aspects of security in Linux:

- User Authentication and Authorization
- File and Directory Permissions
- Firewalls
- Encryption
- Security Auditing
- Network Security
- Logging and Monitoring
- User and Group Management
- Security Policies and Compliance
- Access Controls
- Incident Response
- Backup and Disaster Recovery



SECURITY IN LINUX

Security in Linux is a multi-faceted concept involving various layers of protection to safeguard the operating system, applications, and data from unauthorized access, attacks, and potential vulnerabilities.

Here are key aspects of security in Linux:

- User Authentication and Authorization
- File and Directory Permissions
- Firewalls
- Encryption
- Security Auditing
- Network Security
- Logging and Monitoring
- User and Group Management
- Security Policies and Compliance
- Access Controls
- Incident Response
- Backup and Disaster Recovery



USER AUTHENTICATION AND AUTHORIZATION⁷³

User authentication and authorization are critical aspects of Linux security, ensuring that only authorized users can access the system and that they have the appropriate permissions to perform specific actions.



USER AUTHENTICATION

Password Authentication

- Users typically authenticate using a username and password combination.
- Passwords should be strong and regularly updated.
- Passwords are stored in the `/etc/shadow` file with hashed values.

SSH Key Authentication

- Users can authenticate using public-key cryptography, where a private key is stored on the client, and the public key is added to the `~/.ssh/authorized_keys` file on the server.
- SSH key authentication provides a more secure and convenient way to access systems.

Multi-Factor Authentication (MFA)

- Enforces additional authentication factors, such as a one-time password (OTP) generated by a mobile app or a hardware token.
- Adds an extra layer of security beyond a password.



USER AUTHORIZATION

File and Directory Permissions

- Linux uses a permission system to control access to files and directories.
- Permissions include read (r), write (w), and execute (x) for the owner, group, and others.
- Commands like `chmod` and `chown` are used to modify permissions and ownership.

User Groups

- Users can be organized into groups, and permissions can be assigned to these groups.
- The `groups` command shows a user's group memberships.
- Group membership allows users to share common permissions.

Root (Superuser)

- The root user has superuser privileges, allowing them to perform administrative tasks and access any file on the system.
- Use `sudo` to temporarily elevate privileges for specific commands.



FILE PERMISSIONS AND PROCESS CONTROL

76

In Linux, file permissions determine who can access, modify, or execute a file. They are represented by a combination of three sets of permissions: read, write, and execute, which apply to three categories of users: owner, group, and others.

Read (r): Allows users to view the file's contents.

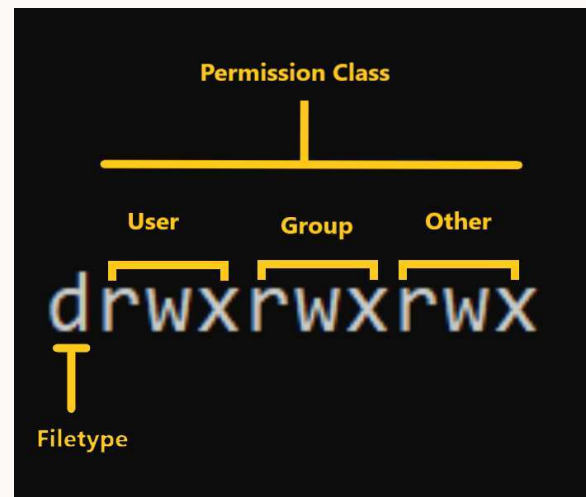
Write (w): Permits users to modify or delete the file.

Execute (x): Grants the ability to run the file if it's a program or script.



PERMISSIONS STRING

77



-rwxrwxrwx a file that everyone can read, write and execute (and delete).

-rw----- a file that only the owner can read and write - no-one else can read or write and no-one has execution rights (e.g. your mailbox file).



FILE PERMISSIONS AND PROCESS CONTROL

78

Representations:

1.Symbolic Representation:

- **r** for read, **w** for write, **x** for execute.
- - denotes the absence of a permission.

Example:

rwxr-xr-- 1 owner group 4096 Feb 28 10:30 sandeep-test

In this example, the owner has read, write, and execute permissions, the group has read and execute permissions, and others have only read permission.

2.Numeric Representation:

- Each permission has a numeric value: read (4), write (2), and execute (1).
- The sum of these values represents the permission set.

Example:

764

- Here the owner has read (4) + write (2) + execute (1) permissions (7), the group has read (4) + write (2), and others have read (4).



CHANGING ACCESS RIGHTS

chmod (changing a file mode)

Only the owner of a file can use **chmod** to change the permissions of a file. The options of **chmod** are as follows

For example, to remove read write and execute permissions on the file **sandeep.txt** for the group and others, type

```
$ chmod go-rwx sandeep.txt
```

Symbol	Meaning
u	user
g	group
o	other
a	all
r	read
w	write (and delete)
x	execute (and access directory)
+	add permission
-	take away permission



USERS AND GROUP

80

In Linux, users and groups are fundamental concepts that help manage and organize system resources and permissions. Here's an overview of users and groups in Linux:

Users:

User Accounts: Each person or process interacting with the system is identified by a unique username, also known as a user account.

UID (User Identifier): Each user is associated with a numerical UID, which uniquely identifies them to the system.

Home Directory: Every user has a home directory where they store their personal files. The home directory is usually located in the /home directory and is named after the user.

Shell: Users interact with the system through a command-line interface called a shell. The default shell for a user is specified in the /etc/passwd file.

Groups:

Group Accounts: Groups are collections of users. They provide a way to organize users with similar access rights and permissions.

GID (Group Identifier): Like users, each group is associated with a numerical GID, which uniquely identifies the group to the system.

Primary and Secondary Groups: Each user belongs to a primary group, and can also be a member of several secondary groups. The primary group is specified in the /etc/passwd file, while secondary groups are listed in the /etc/group file.



USERS AND GROUP

81

Important Files:

- **/etc/passwd:** Contains information about user accounts, including username, UID, primary GID, home directory, and default shell.
- **/etc/shadow:** Stores encrypted user passwords and other password-related information.
- **/etc/group:** Contains information about group accounts, including group name, GID, and a list of user names that belong to the group.
- **/etc/gshadow:** Similar to /etc/shadow but for groups. It stores encrypted group passwords and other group-related information.



USERS AND GROUP

82

To get the list of users and groups on a Linux system, you can use the following commands:

1. List Users: To list all users on the system, you can use the cat command with the /etc/passwd file

`cat /etc/passwd` OR `getent passwd`

2. List Groups: To list all groups on the system, you can use the cat command with the /etc/group file

`cat /etc/group` OR `getent group`

